SIMATS ENGINEERING

ASSESSMENT TOOL 3 — MOCK INTERVIEW ANSWER KEY

Course: Database Management Systems (CSA05)

Course Outcome: CO5 — Query processing, optimization, and transaction management (BL3, BL4)

Activity Tool: Mock Interview (Low) | Weightage: 30%

Total Marks: 50 (10 × 5 marks)

Code of Conduct

I **P Shashank** Reg. No: **192524282** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Shashank

Each answer below is written as a candidate's spoken interview response, followed by a short executed demonstration illustrating the concept, shown as a terminal screenshot.

> **Q1. Interviewer:** "*Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?*"
>
> *BL3 – Apply | Marks: 5*

Candidate Answer

Query processing has four broad stages. First, parsing checks the SQL is syntactically correct and builds a parse tree; the validator then confirms every table and column actually exists in the catalog and types line up. Next, that tree is translated into a logical plan expressed in relational algebra. The optimizer then generates several equivalent physical plans — different join orders, join algorithms, and access paths (index seek vs. full scan) — and estimates the I/O and CPU cost of each using statistics like table cardinality, index selectivity, and available memory. It picks the cheapest one. Finally, the execution engine runs that chosen plan and streams the results back.

The optimizer decides using a cost model: it estimates the number of page/block I/Os (and sometimes CPU cost) each candidate plan would need, based on stored statistics, and picks the plan with the lowest total estimated cost — not necessarily the plan that looks simplest.

Executed Demo

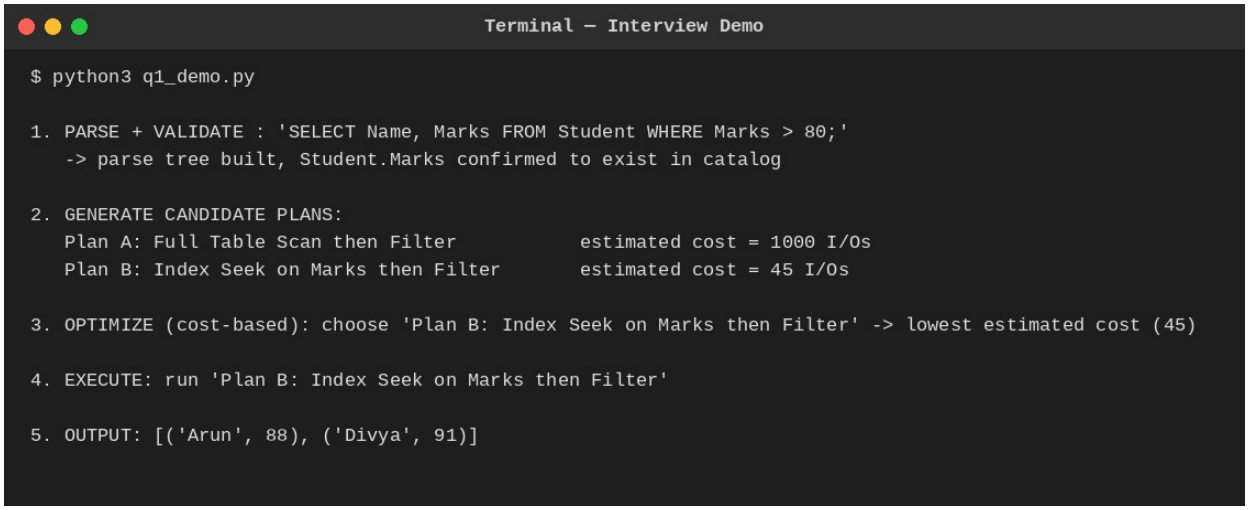

Fig. Q1 — Two candidate plans costed; optimizer selects the cheaper index-based plan.

> **Q2. Interviewer:** "*What is the difference between heuristic optimization and cost-based optimization? When would you use each?*"
>
> *BL4 – Analyze | Marks: 5*

Candidate Answer

Heuristic optimization applies fixed rules that are almost always beneficial, regardless of actual data — for example, always pushing a selection or projection as early as possible in the query tree so later operations work on fewer rows. It needs no statistics and is cheap to apply. Cost-based optimization instead estimates the actual I/O/CPU cost of multiple candidate plans using table statistics (row counts, index selectivity, available buffer pages) and picks whichever is numerically cheapest — this is essential for decisions like join order and join algorithm, where the 'obviously good' choice can flip depending on table sizes.

In practice, both are used together: the optimizer first applies heuristic rewrites to shrink the search space and the intermediate data sizes, then applies cost-based comparison among the remaining candidate plans for the genuinely data-dependent decisions.

Executed Demo

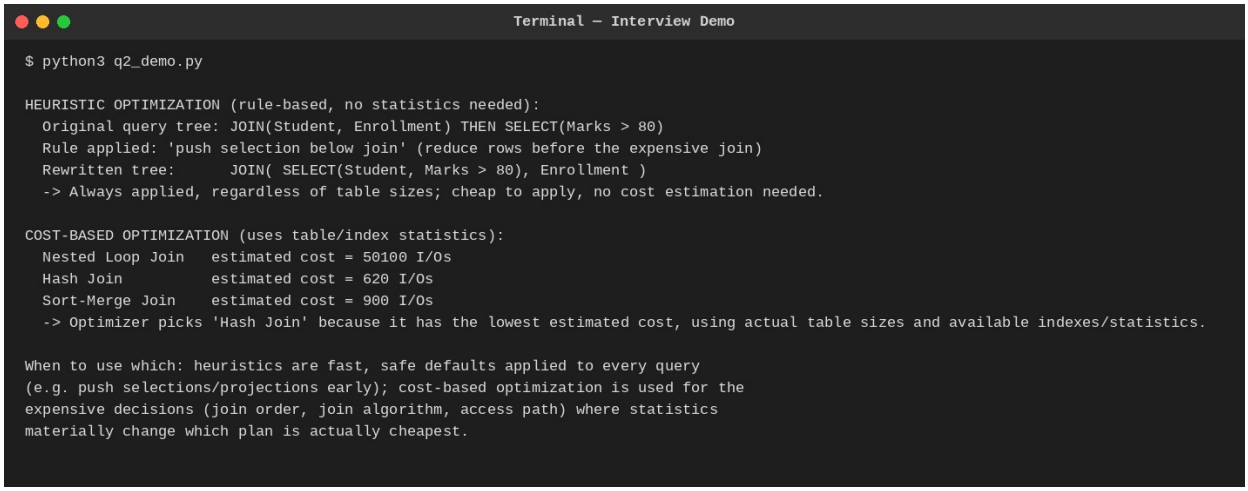

Fig. Q2 — A heuristic rewrite (selection push-down) alongside a cost-based join comparison.

> **Q3. Interviewer:** "*How would you implement database connectivity in a real project? Explain the difference*

between JDBC and ODBC."

BL3 – Apply | Marks: 5

Candidate Answer

In a real project I'd use a connection pool (not one raw connection per request) so the app reuses a small set of already-open connections instead of paying the TCP/authentication handshake cost every time. Application code borrows a connection, runs its query inside a transaction, commits or rolls back, and always returns the connection to the pool in a finally block so a failure never leaks it.

JDBC (Java Database Connectivity) is a Java-specific API: a pure-Java driver talks to the database directly over its native network protocol, so no OS configuration is needed — just the driver JAR on the classpath. ODBC (Open Database Connectivity) is a language-neutral C-based API: it goes through an OS-registered Driver Manager and a configured Data Source Name (DSN), which lets many languages (C/C++, and others via wrapper libraries) share the same underlying driver, at the cost of needing that DSN set up on every machine that connects.

Executed Demo

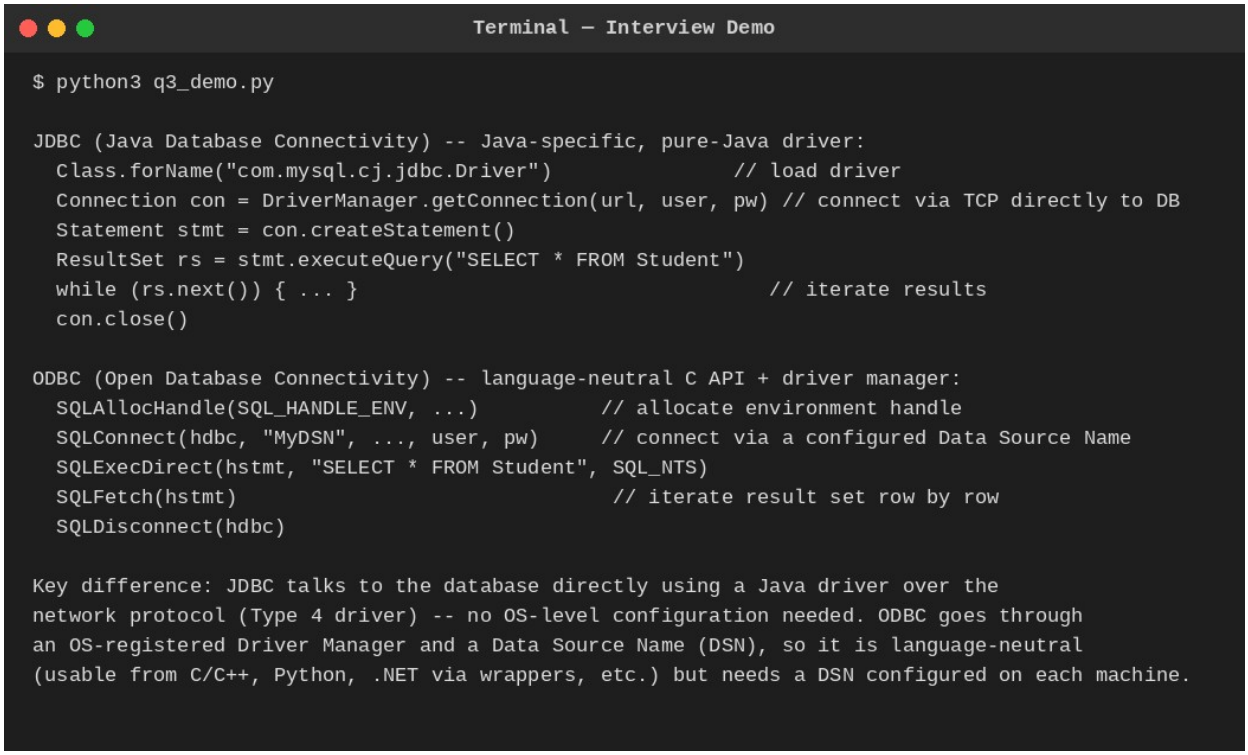

Fig. Q3 — JDBC's direct driver connection vs ODBC's Driver-Manager/DSN flow.

Q4. Interviewer: *"What are ACID properties? Give a real-world example where each property is critical."*

BL3 – Apply | Marks: 5

Candidate Answer

- Atomicity — a transaction is all-or-nothing. Example: a bank transfer must not leave money debited from one account without it being credited to the other; if the system crashes mid-transfer, the whole operation rolls back.
- Consistency — a transaction moves the database from one valid state to another, respecting all constraints. Example: an airline seat-booking system must never end up with two confirmed passengers assigned the same

seat.

• Isolation — concurrent transactions don't see each other's uncommitted changes. Example: two customers booking the last concert ticket simultaneously must not both succeed because each read a stale 'available' count.

• Durability — once committed, changes survive a crash. Example: after an ATM dispenses cash and confirms the withdrawal, that debit must still be recorded even if the server crashes one second later.

Executed Demo

```
Terminal — Interview Demo
$ python3 q4_demo.py

Initial balances: {'A': 1000, 'B': 500}

-- Scenario 1: successful transfer (all 4 ACID properties hold) --
BEGIN TRANSACTION: transfer 200 from A to B
  debit A  -> 800
  credit B -> 700
COMMIT: both updates made permanent (Durability)
Balances after: {'A': 800, 'B': 700}  (Consistency: total is still 1500)

-- Scenario 2: failure occurs after debit, before credit --
BEGIN TRANSACTION: transfer 300 from A to B
  debit A  -> 500
  ERROR: simulated crash after debit, before credit
ROLLBACK -> balances restored to {'A': 800, 'B': 700} (Atomicity preserved)
Balances after: {'A': 800, 'B': 700}  (Atomicity: partial debit was rolled back, total still 1500)

Isolation (not shown by this single-threaded demo): a concurrent transaction reading
A's balance mid-transfer must not see the intermediate, partially-debited value.
```

Fig. Q4 — Bank transfer: successful commit, then a mid-transfer failure rolled back atomically.

> **Q5. Interviewer:** "*Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?*"
>
> *BL4 – Analyze | Marks: 5*

Candidate Answer

• Dirty read — a transaction reads a value written by another transaction that hasn't committed yet; if that writer later aborts, the reader has used a value that never really existed.

• Lost update — two transactions read the same value, both compute a new value independently, and the second write silently overwrites the first, losing that update entirely.

• Unrepeatable read — a transaction reads the same row twice and gets two different values because another transaction committed a change to that row in between.

Two-Phase Locking solves all three by making a transaction acquire a lock before touching data and hold it until it stops acquiring new locks (the shrinking phase). A writer's exclusive lock blocks any other transaction from reading or writing that item until it commits, which prevents dirty reads and lost updates; holding read locks until the shrinking phase similarly prevents another transaction from changing a value out from under a still-active reader, addressing unrepeatable reads.

Executed Demo

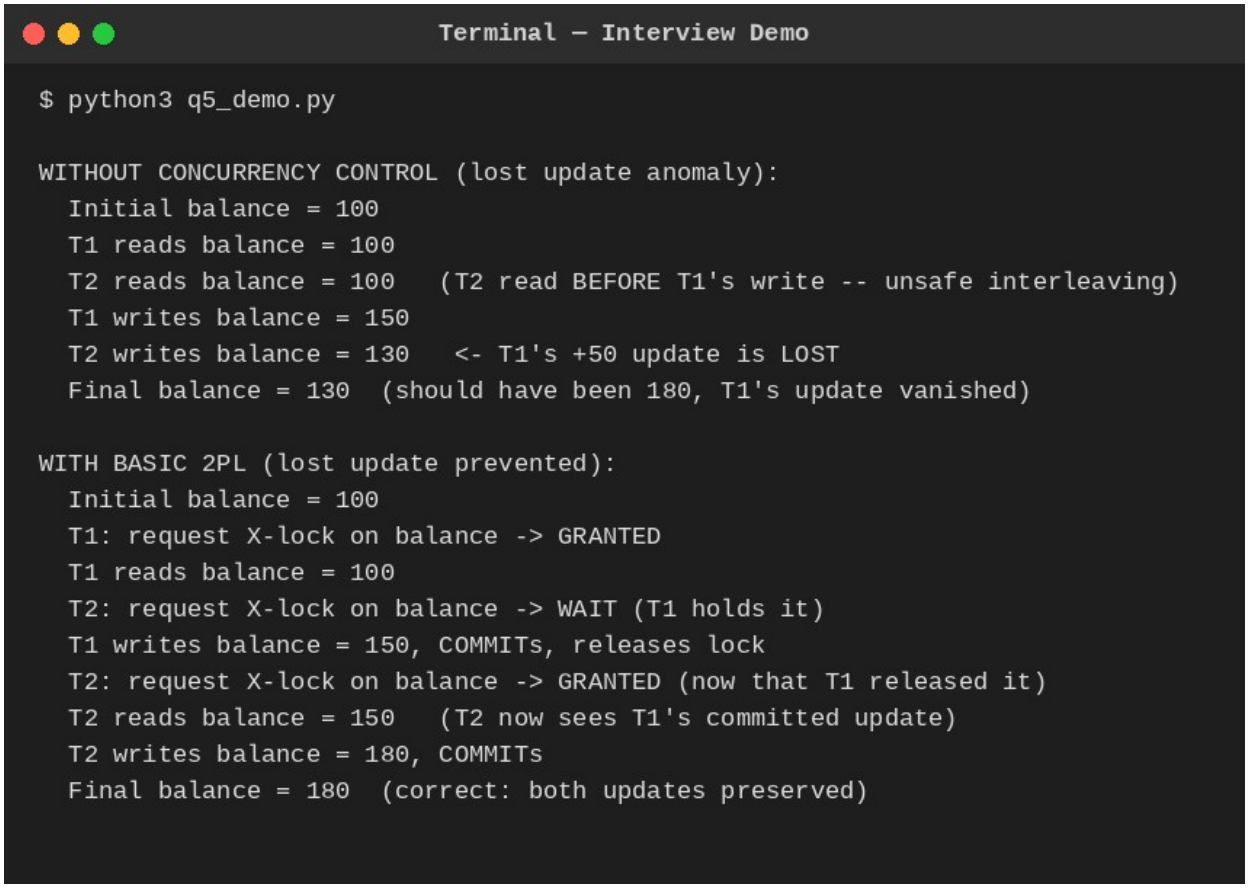

Fig. Q5 — Lost update occurring without locking, then prevented once 2PL is applied.

> **Q6. Interviewer:** *"What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies."*
>
> *BL4 – Analyze | Marks: 5*

Candidate Answer

A deadlock is a cycle of transactions each waiting for a lock held by the next one in the cycle, so none of them can ever proceed. Detection is done by maintaining a Wait-For Graph — an edge Ti → Tj means Ti is waiting on a lock held by Tj — and periodically running a cycle-detection DFS over it; a back-edge to a node still on the current recursion stack proves a cycle exists, and the system aborts one transaction from that cycle (the victim) to break it.

Two prevention strategies avoid deadlocks before they happen, using transaction timestamps: Wait-Die lets an older transaction wait for a younger one, but forces a younger transaction requesting a lock held by an older one to abort ('die') and restart; Wound-Wait does the reverse (older 'wounds'/pre-empts the younger). A simpler, non-graph-based approach is timeout-based prevention: if a transaction waits longer than a threshold, the system just assumes a deadlock and aborts it.

Executed Demo

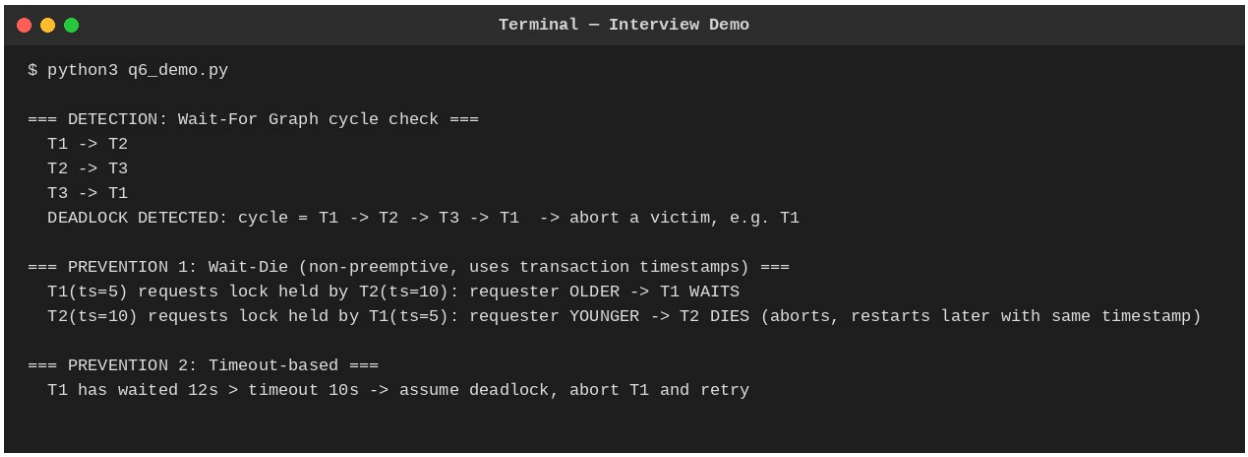

Fig. Q6 — WFG cycle detection, plus Wait-Die and timeout-based prevention.

> **Q7. Interviewer:** "*Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?*"
>
> *BL4 – Analyze | Marks: 5*

Candidate Answer

Immediate Update writes a transaction's changes to disk as soon as they happen, even before commit, so recovery after a crash needs both a REDO pass (for committed transactions, in case a write hadn't reached disk yet) and an UNDO pass (for uncommitted transactions, in case a write had reached disk). Deferred Update instead buffers all of a transaction's writes in memory and only flushes them to disk after commit, so an uncommitted transaction's writes are never on disk at all — recovery only ever needs a REDO pass, and UNDO is unnecessary by construction.

Deferred Update is preferred when transactions are short and their write sets fit comfortably in memory, since it simplifies recovery. Immediate Update is preferred for long-running transactions with large write sets, since buffering everything until commit would consume too much memory; it accepts the extra complexity of UNDO logging in exchange for better scalability.

Executed Demo

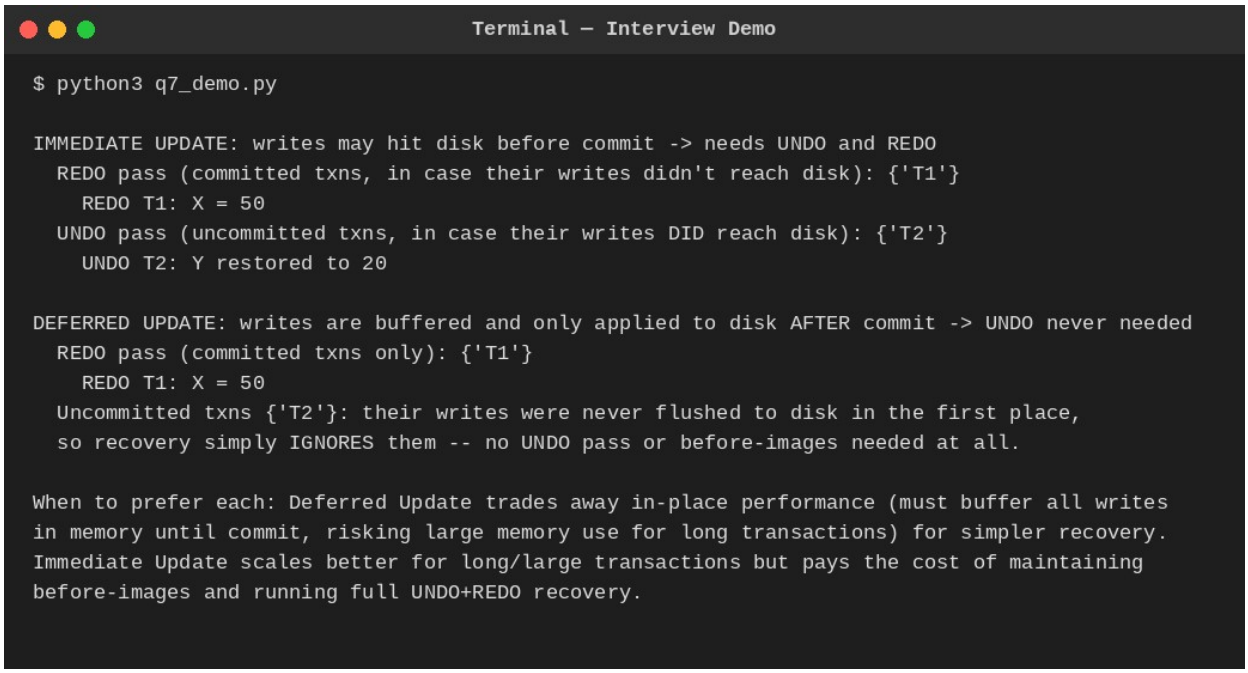

Fig. Q7 — Same log recovered with Immediate Update (REDO+UNDO) vs Deferred Update (REDO only).

Q8. Interviewer: "*Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?*"

BL3 – Apply | Marks: 5

Candidate Answer

Shadow paging never updates a page in place. A transaction works on a copy of the page table (the current table); any page it writes is copied to a brand-new frame on disk, and only the current table's pointer for that page is updated — the shadow (old) table still points to the untouched original frames. Commit is a single atomic operation: flush the current table to disk, then swap the shadow pointer to point at it.

- Advantage: no logging is needed at all, and recovery is instant — a crash before commit just means discarding the (still-unreferenced) current table; the old shadow table is already a fully consistent database.
- Limitation: it fragments the database on disk (updated pages scatter across new frames) hurting locality/performance over time, garbage-collecting orphaned old frames is extra work, and it doesn't naturally support fine-grained concurrent transactions the way log-based, record-level locking does — log-based recovery (e.g. ARIES) is far more common in real systems for that reason.

Executed Demo

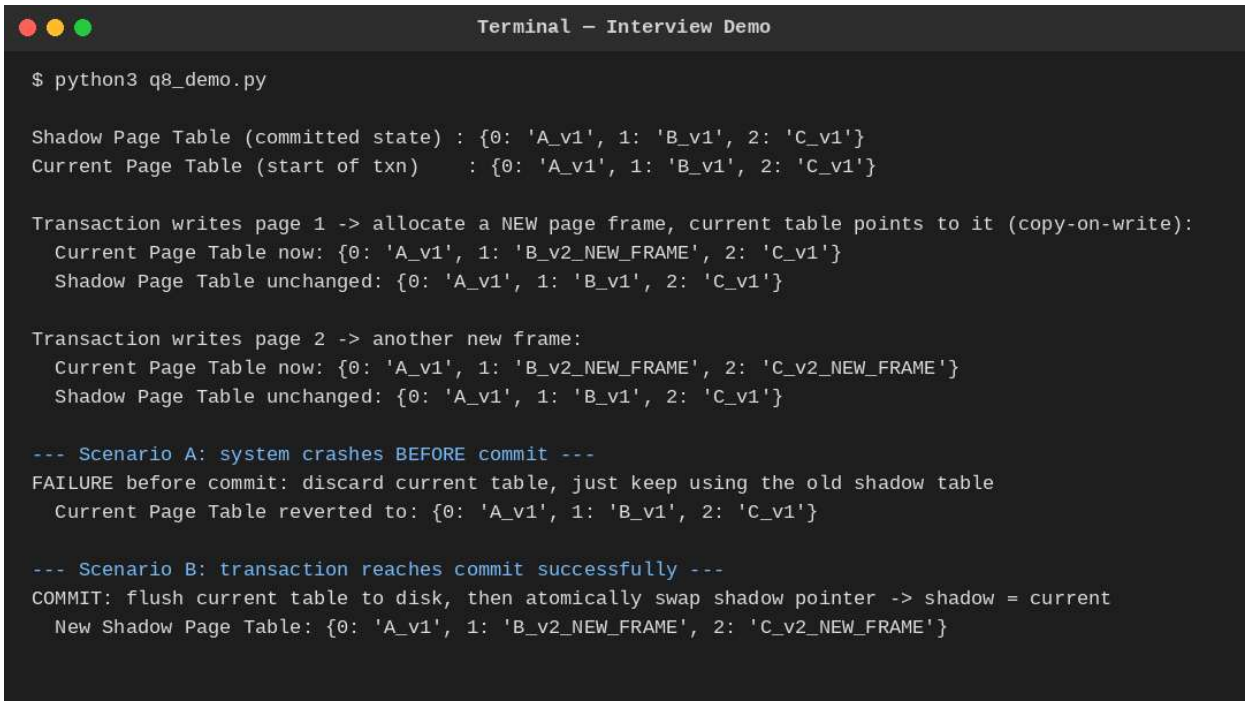

Fig. Q8 — Copy-on-write page allocation, atomic commit swap, and pre-commit crash rollback.

> **Q9. Interviewer:** "*What is a serializability schedule? How do you test for conflict serializability using a precedence graph?*"
>
> *BL4 – Analyze | Marks: 5*

Candidate Answer

A schedule is conflict serializable if it is equivalent, in terms of the relative order of all conflicting operations, to some serial (one-transaction-at-a-time) execution of the same transactions — meaning it's safe to run concurrently because it produces the same effect as some valid serial order.

To test it, build a precedence graph: one node per transaction, and a directed edge Ti → Tj whenever Ti has an operation that conflicts with (and precedes) an operation of Tj on the same data item — conflicts being any pair of operations on the same item where at least one is a write. The schedule is conflict serializable if and only if this graph is acyclic; a DFS-based cycle check finds this in O(V+E) time, and if acyclic, any topological order of the graph gives a valid equivalent serial order.

Executed Demo

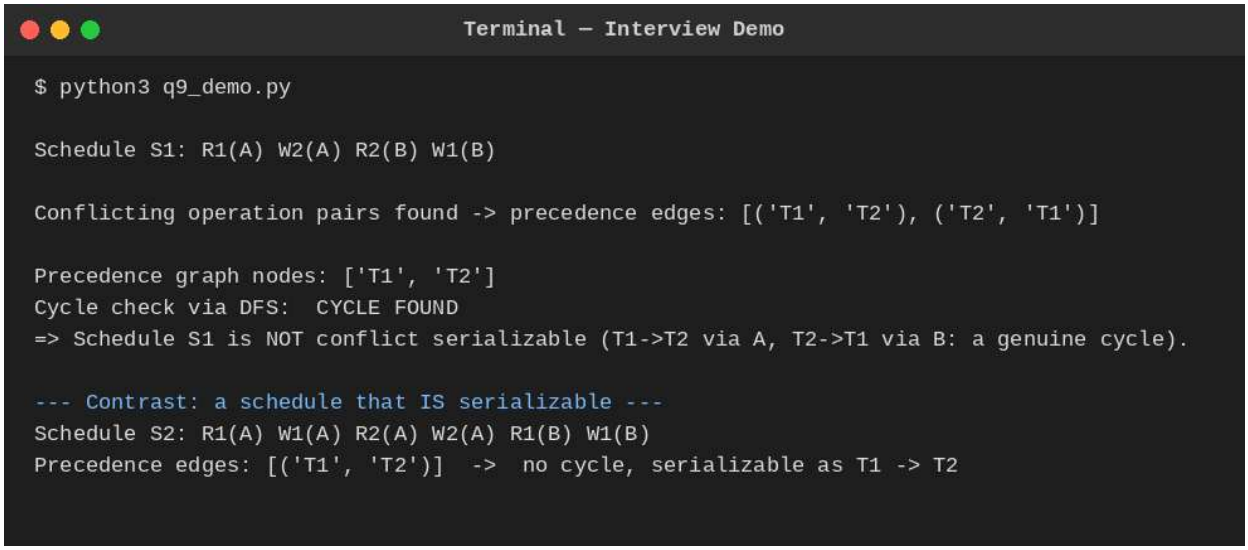

Fig. Q9 — A cyclic (non-serializable) schedule contrasted with an acyclic (serializable) one.

> **Q10. Interviewer:** "*Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?*"
>
> *BL4 – Analyze | Marks: 5*

Candidate Answer

- Basic 2PL — locks may be released as soon as the shrinking phase begins, which can be before commit. This guarantees serializability but allows cascading aborts: if a transaction reads a value another transaction wrote and released the lock on early, and that writer later aborts, the reader must abort too.
- Strict 2PL — exclusive (write) locks are held until commit or abort, though shared (read) locks may still be released earlier. No other transaction can ever see an uncommitted write, which eliminates cascading aborts.
- Rigorous 2PL — every lock, shared or exclusive, is held until commit or abort. This is the simplest to reason about, since transactions effectively appear to execute in their commit order.

Strict 2PL is the most commonly used in practice — it's the default locking discipline behind most commercial engines (e.g. MySQL InnoDB, Oracle). It gets the practically important benefit (no cascading aborts) without giving up the extra concurrency of releasing read locks a little earlier than Rigorous 2PL would.

Executed Demo

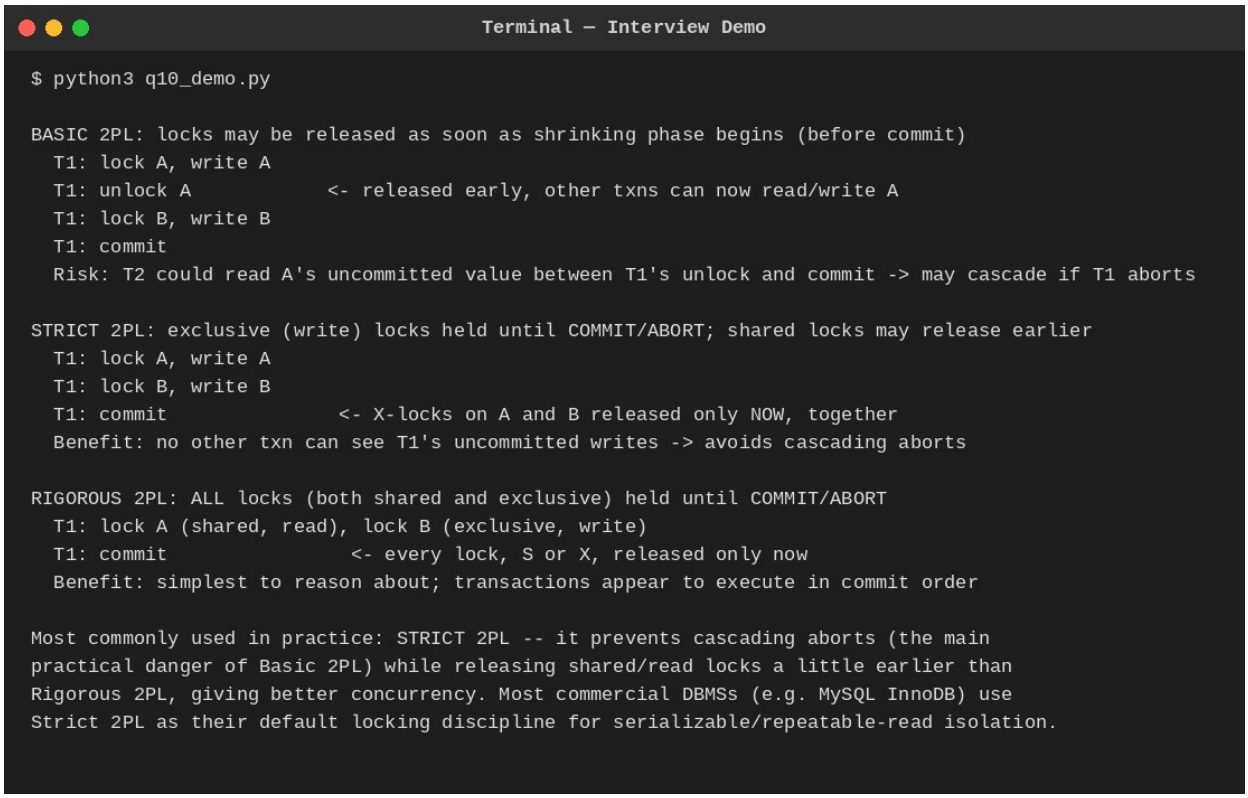

Fig. Q10 — Lock release timing compared across Basic, Strict, and Rigorous 2PL.

End of Answer Key